



UNIVERSITI SAINS MALAYSIA

School of Electrical and Electronics Engineering
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

EEM 441

Control & Instrumentation Laboratory

Experiment 8

TEMPERATURE CONTROL SYSTEM

Date : _____

Group No. : _____

Group Members : _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

Experiment 8

Temperature Control System

Objectives

1. To design and build a signal-conditioning circuit for an NTC thermistor.
2. To interface the conditioned temperature signal to a Raspberry Pi via an ADC0804 analog-to-digital converter.
3. To implement a computer-based system that displays the measured temperature and performs ON/OFF control of an indicator LED via Raspberry Pi GPIO.

Equipment / Software Required

- NTC thermistor and trainer board with indicator LED
- Components for the student-designed signal-conditioning circuit (op-amps, resistors, capacitors)
- ADC0804
- Raspberry Pi 4B
- Breadboard and jumper wires
- Digital multimeter
- Temperature sensor

Introduction

In many industrial processes, temperature must be regulated automatically. Two general strategies are used: *logic (ON/OFF) control*, in which a heater, cooler, or indicator is simply switched on or off when a threshold is crossed, and *fuzzy/proportional control*, which requires a more elaborate definition of “hot” or “cold” before acting. This experiment uses logic control.

An NTC (negative temperature coefficient) thermistor is a resistor whose resistance decreases as its temperature increases. Given the thermistor’s resistance–temperature characteristic (from its datasheet), a measured resistance can be converted to a temperature, and vice versa.

The signal path for this experiment is: the thermistor’s resistance is converted to a voltage by a signal-conditioning circuit (which the student must design), this voltage is digitised by the ADC0804 and read by the Raspberry Pi via GPIO through a level shifter, and software

on the Pi computes the corresponding temperature, displays it, and drives the indicator LED on the trainer according to a threshold comparison (Figure 8.1).

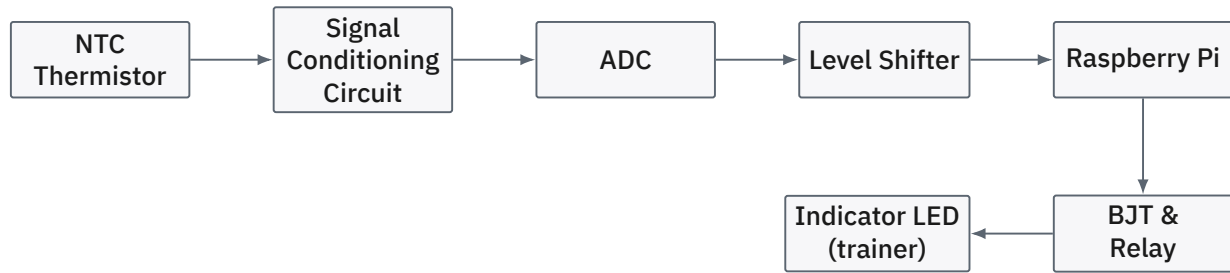


Figure 8.1: Block diagram of the temperature measurement and control system.

Procedure

Activity 1 – Signal-conditioning circuit design

Objective: design and build a circuit that converts the thermistor's resistance to a voltage suitable for the ADC0804 input.

1. Obtain the resistance–temperature characteristic for the NTC thermistor from its datasheet, and determine the resistance range corresponding to the temperature range you intend to measure and control.
2. Design a signal-conditioning circuit that converts the thermistor's resistance to a voltage. Calculate your component values and justify your design choices, including the expected output voltage range over your chosen temperature range. Keep the output within the ADC0804's usable input range.
3. Include a voltage-follower (buffer) stage to eliminate loading effects between the conditioning circuit and the ADC0804 input.
4. Build the circuit and verify its output voltage against a reference temperature measurement at two or more points, and check linearity/consistency with your design calculations.

Activity 2 – Interfacing to the Raspberry Pi via the ADC0804

Objective: connect the conditioned temperature signal to the Raspberry Pi via the ADC0804 and a level shifter, and connect the LED control line to a GPIO output.

1. Connect the output of your signal-conditioning circuit to the V_{in} inputs of the ADC0804.

2. Connect the ADC0804's D0–D7 and $\overline{\text{INTR}}$ outputs through the level shifter to Raspberry Pi GPIO input pins, and the $\overline{\text{CS}}$, $\overline{\text{RD}}$, $\overline{\text{WR}}$ control lines to Raspberry Pi GPIO output pins. **Do not connect the ADC0804's 5 V logic outputs directly to the Pi.**
3. Connect the trainer's indicator LED control line to a Raspberry Pi GPIO output pin, via a transistor (BJT) or relay driver stage if the LED/trainer input requires more current than a GPIO pin can safely source directly (consult the trainer's LED driver requirements before connecting).
4. Verify both the input and output wiring with a simple test script before writing the full control program: confirm a static ADC reading is read back correctly, and confirm the LED can be toggled on and off from GPIO.

Activity 3 — Temperature monitoring and LED control software

Objective: implement software that displays the measured temperature and controls the LED according to a threshold.

1. In Python or C/C++, write a program that:
 - Periodically triggers an ADC0804 conversion and reads back the digitised thermistor voltage.
 - Converts the reading to a temperature using your circuit's calibration from Activity 1.
 - Displays the current temperature in a simple terminal or GUI display, updating in real time.
 - Compares the measured temperature against the threshold and sets the LED GPIO output accordingly.
2. For GPIO access, either library introduced in Experiment 7 is suitable here: `RPi.GPIO` or `gpiozero` in Python, or `libgpiod/wiringPi` in C/C++. Since this experiment only needs periodic polling (not precise edge timing), a simple timed-loop read is sufficient — interrupt-driven GPIO is not required here.
3. Report and justify your design choice for the signal-conditioning circuit.
4. Use the flowchart in Figure 8.2 as a starting point for your control logic, adapting the ON/OFF branch to match your chosen design.

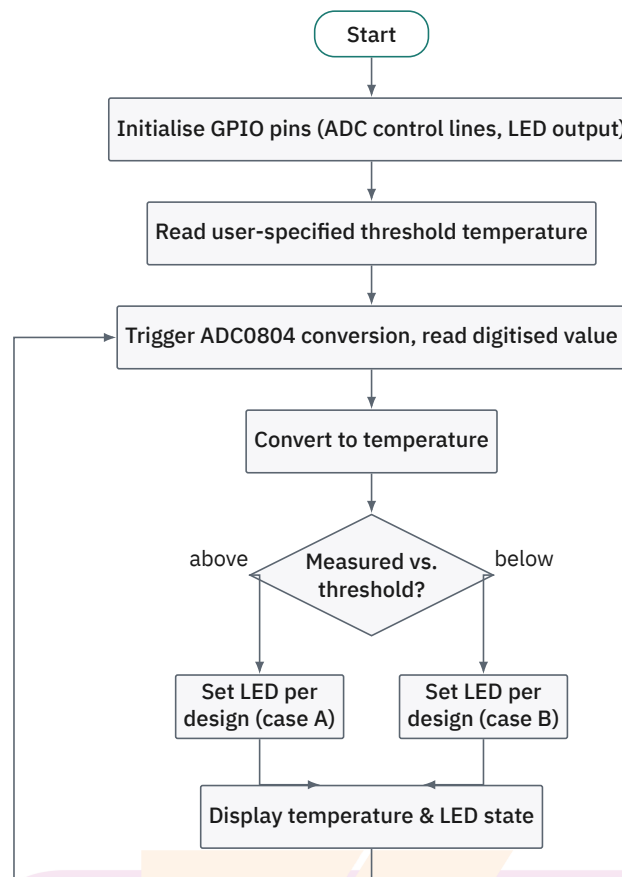


Figure 8.2: Temperature monitoring and LED control loop.

- Set the controlled (threshold) temperature to a chosen value and record the measured temperature and LED state at 1-minute intervals for 35 minutes (Table 8.1). Plot temperature against time.
- Repeat for at least two further threshold temperatures. Plot your results.
- Discuss your findings, including the responsiveness of the LED control and the accuracy of your temperature conversion.

Table 8.1: Temperature and LED-state log.

Threshold temp.	Time (min)	Measured temp.	LED state
	1		
	2		
	...		
	35		

Report

- Present your signal-conditioning circuit design (topology, calculations, component values) and its calibration against a reference temperature measurement.
- State and justify your chosen LED ON/OFF logic (Activity 3, step for control logic).
- Present and discuss your temperature-vs-time plots for each threshold tested with respect to the errors.

4. What sources of error or lag affect the accuracy and responsiveness of this system (thermistor thermal mass, signal-conditioning bandwidth, ADC conversion time, software polling rate, etc.)?

